CSE440 *Group-10* *Section-01*

Md. Istiaque Khan *Rifa Rahanuma* *Ariful Alam Abir* *Mahir Waiz Miel*
ID: 2221244642 *ID: 2221293042* *ID: 2122236642* *ID: 2232474642*

Self-Driving Car Racing Game Using Deep Reinforcement Learning

Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh

Abstract

This project creates a two-dimensional racing game in which an agent trained using Reinforcement Learning is required to learn to drive through its interaction with an environment simulated. The interface is provided through Pygame, while NumPy, Pillow, PyTorch, and Matplotlib provide all the necessary abstraction for processing the track, implementing the Deep Q-Network, and visualizing the results. The vehicle uses five wall-distance sensors and its speed to pick either one of the four actions: go left, go right, drive straight, and stop. The system allows for three levels of difficulty for the track, namely Double DQN, experience replay, ε-greedy exploration, target networks, checkpoint saving/loading, deterministic evaluation, and automated testing. The best run on the Easy track has shown that the agent achieved a maximum reward of 849.0, and the model used has finished all the episodes without a collision.

Key Words: Deep Reinforcement Learning, Deep Q-Network, DQN, Autonomous Driving, Ray-Cast Sensors, Pygame, Reinforcement Learning, Q-Learning.

Introduction

Reinforcement learning (RL) allows an agent to improve its behavior through interaction and rewards instead of being given the correct action for every situation. A racing game provides a useful educational environment because the agent repeatedly senses the road, chooses an action, observes the vehicle's movement, receives a reward, and updates its policy. The project is not intended to be a complete driving simulator. Instead, it focuses on making agent environment interaction, state and action design, reward engineering, exploration, value approximation, training, saving, and evaluation easy to understand and explain on a normal laptop CPU. The system follows a clear separation between the game layer and the AI layer. The game layer handles Pygame rendering, vehicle kinematics, and track collision, while the AI layer handles the DQN agent, replay buffer, and trainer. This separation also allows headless training at around 16×–128× real time speed without rendering overhead, while still allowing the agent to be visualized during live runs.

Objectives and Scope

1. Build a playable 2D racing game with simple vehicle physics. 2. Use image masks for road collision and five ray distance sensors. 3. Represent the state as five sensor values plus speed (6 dimensional vector). 4. Implement four

discrete actions and a specified reward table. 5. Train a 6–128–128–4 Double DQN with replay memory and epsilon-greedy exploration. 6. Save/load model checkpoints, display live metrics, and generate evaluation graphs. 7. Keep training CPU friendly and verify behavior with automated tests.

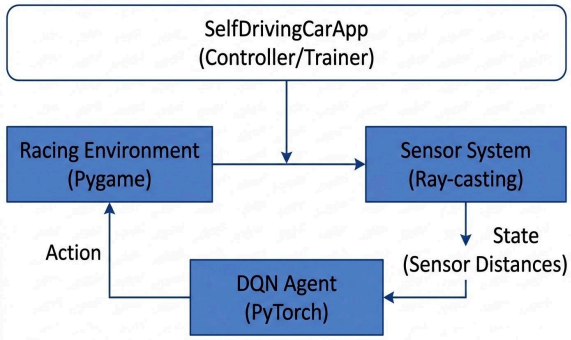
Included scope: One car, manual and AI modes, Easy/Medium/Hard procedurally generated tracks, three car colors (red, blue, green), live and headless training, saved models, raw metrics in CSV/JSON, and 20 automated tests.

Excluded scope: Realistic tire/suspension physics, real road perception, multiple competing AI cars, and safety critical vehicle control.

Technology Selection

Part	Technology	Purpose & Version
Main Implementation	Python	Core language (v3.13)
Game Interface	Pygame	Window, controls & rendering (v2.6)
Neural Network	PyTorch	DQN & checkpoints (v2.x)
Numeric Processing	NumPy	State arrays & math
Track Images	Pillow (PIL)	PNG & collision mask processing
Graphs	Matplotlib	Plots for reward, loss & lap time
Testing	Pytest	Automated system verification

System Architecture



The system uses a **Model View Controller (MVC)** structure to separate the game, learning, and display components.

Controller — `SelfDrivingCarApp`

- Manages **Main Menu, Manual Drive, Live Training, and Evaluation**.
- Handles user input, mode changes, and simulation speed (**1×–128×**).

Model — `RacingEnv + DQNAgent`

- `RacingEnv`: handles **vehicle movement, sensor readings, collision detection, and reward calculation**.
- `DQNAgent`: handles **action selection, replay buffer, policy network, target network, and Double DQN learning**.
- Uses **5 sensor rays + speed → 6D state vector**.

View — `PygameRenderer`

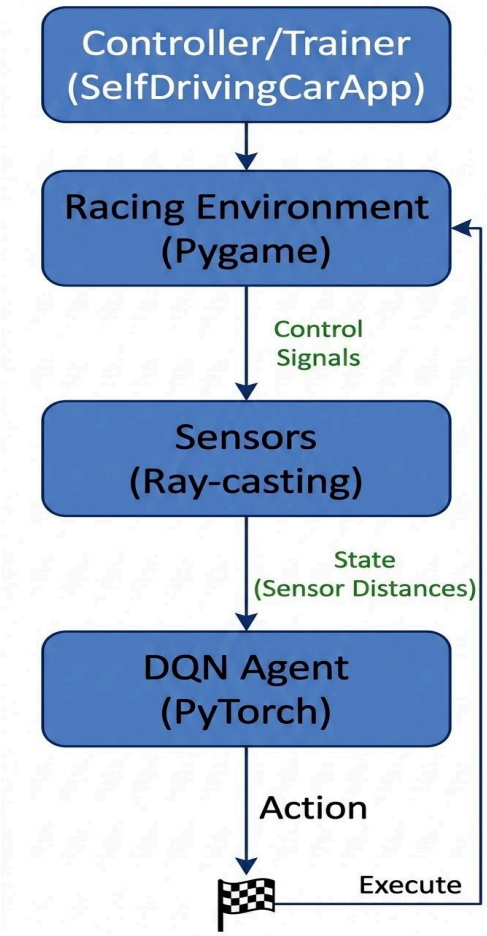
- Displays the **track, car, sensor rays, reward, loss, and ε value**.
- Can be disabled during headless training to reduce rendering overhead.

Training Flow:

Sense → Decide → Act → Learn → Sync

`5 Rays + Speed → 6D State → ε-Greedy Action → Vehicle Update → Replay + Double DQN → Target Network Sync`

Figure: The architecture separates the learning components from rendering, allowing the same trained model to run in both **headless training** and **live Pygame evaluation** without changing the main learning code.



Game Environment

Vehicle Physics

The car stores position (x, y), heading angle θ, speed v, and total distance. The screen coordinate system uses positive x rightward and positive y downward. Position and velocity are updated each simulation step:

1. $x(t+1) = x(t) + \cos(\theta(t)) \cdot v(t)$
2. $y(t+1) = y(t) + \sin(\theta(t)) \cdot v(t)$
3. $\theta(t+1) = \theta(t) + s \cdot \delta\_steer \cdot \sigma(v)$
4. $v(t+1) = \text{clip}(v(t) + a \cdot \alpha - b \cdot \beta - f, 0, v\_max)$

where $\delta\_steer = 4.2°$, $\alpha = 0.20$ (acceleration), $\beta = 0.34$ (brake power), $f = 0.035$ (rolling friction), and $v\_max = 7.0$ units/step. The steering function $\sigma(v)$ scales with speed: $\sigma = 0.38 + 0.62 \cdot (v / v\_max)$, preventing unrealistic high-speed turning. The car body dimensions are 30.0 × 16.0 units.

Image-Based Tracks and Collision

Each difficulty level has a colored display PNG, a grayscale white road/black wall mask, and JSON metadata containing a 720 point centerline, twelve ordered checkpoints, start pose, and road width. Collision is detected by testing nine probe points around the rotated car body against the binary mask: all nine must land on white (road) pixels for the car to remain alive. Three procedurally generated tracks provide progressive difficulty:

- **Easy:** Wide oval with gentle curves.
- **Medium:** Irregular S-curves requiring dynamic braking.
- **Hard:** Tight hairpin turns with narrow road width.

Dynamic Traffic Obstacles

To prevent the agent from memorizing a single static trajectory, up to 3 traffic vehicles are spawned along the track centerline at random lateral offsets (±30 units from center) with random heading perturbations (±45°). These act as stationary obstacles that the agent's sensors must detect and avoid. Sensor rays terminate upon hitting a traffic vehicle within a 23.0-unit proximity radius, and direct car to traffic proximity below 22.0 units triggers a collision.

Progress and Checkpoints

The environment performs a local nearest centerline search around the previous index, preventing collision near another track section

from producing an impossible progress jump. Twelve ordered absolute checkpoint targets prevent repeated reward from oscillating across one checkpoint line.

State, Actions, and Rewards

State Representation

Five rays are cast at −90°, −45°, 0°, +45°, and +90° relative to the car heading.
Each ray steps forward in increments of 2.0 units until it hits a wall pixel, a traffic vehicle, or reaches the maximum sensor distance of 165.0 units.
Each distance and the current speed are normalized to [0, 1].

state = [d_L/d_max, d_FL/d_max, d_F/d_max, d_FR/d_max, d_R/d_max, v/v_max]

resulting in a compact 6-dimensional continuous state vector.

Action Space

Action	Throttle	Steering	Brake
Turn Left	1	−1	0
Turn Right	1	+1	0
Go Straight	1	0	0
Brake	0	0	1

Reward Function

Reward engineering provides both local guidance and long term goals:

Situation	Reward
Driving forward	+1.0
Passing a checkpoint	+20.0
Completing a lap	+100.0
Collision (terminal)	−100.0
Driving backwards	−10.0
Standing still	−2.0

Deep Q-Network

Network Architecture

The policy network is a fully connected MLP with six inputs, two hidden ReLU layers of 128 units each, and four Q-value outputs. Weights are initialized using Kaiming uniform initialization. A separate target network with identical architecture stabilizes the Bellman target.

`6 inputs → 128 ReLU → 128 ReLU → 4 Q-values`

Double DQN Update Rule

For each sampled transition (s, a, r, s', done), the target is computed using Double DQN [3] to mitigate overestimation bias:

$$y = r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{target}}(s', \text{argmax}_{a'} Q_{\text{policy}}(s', a'))$$

Action selection uses the policy network while action evaluation uses the target network.

Training Stabilization Techniques

1. **Experience Replay:** A circular buffer of capacity 50,000 stores transitions. Random mini batches of 128 are

sampled every 4 environment steps, breaking temporal correlation.
2. **Epsilon Greedy Exploration:** ε decreases linearly from 1.0 to 0.05 over 45,000 steps.
3. **Smooth L1 (Huber) Loss:** Limits sensitivity to large temporal difference errors.
4. **Adam Optimizer:** Learning rate $\alpha = 1.0 \times 10^{-3}$.
5. **Gradient Clipping:** Gradient norm clipped at 10.0 to prevent exploding gradients.
6. **Target Network Sync:** Target weights copied from policy every 600 environment steps.

Training Configuration

Hyperparameter	Value
Training episodes	500–1,000
Discount factor (γ)	0.99
Learning rate (α)	1.0×10^{-3}
Batch size	128
Replay capacity	50,000
Replay warm-up	750 transitions
Train every N steps	4
Target network update	600 env steps
ε range	1.0 → 0.05
ε decay steps	45,000
Hidden layer size	128
Gradient clip norm	10.0
Double DQN	Enabled
Device / Seed	CPU / 440

The exact run configuration is stored as JSON alongside the result data. Headless training uses the same environment as the Pygame interface but avoids rendering overhead, allowing speeds of up to 128× real time.

Results

Training Summary (Easy Track, Best Run)

The most successful Easy-track training run spanned 1,000 episodes. During the high-ε exploratory phase (first ~500 episodes), the agent collided in nearly every episode, producing a mean reward of −47.0 over the first 100 episodes. As ε decayed below 0.4, reward began climbing. The agent first completed a full lap at episode 715.

Metric	Value
Training episodes	1,000
Exploratory lap completions	26 (2.6%)
Best single-episode reward	849.0
Mean reward (completed episodes)	766.7
Mean completed lap length	441 steps
Collision-terminated episodes	974
Mean reward (first 100 episodes)	−47.0
Mean reward (last 100 episodes)	+134.3
Initial ε	0.998
Final ε	0.05

Curriculum Learning for Medium and Hard Tracks

Direct from scratch training on the Hard track over 1,000 episodes produced zero completed laps (0.0% success rate, best reward 263.0). The tight hairpin turns caused the agent to collide before accumulating meaningful positive reward. We addressed this via Curriculum Learning: the Easy track best checkpoint was loaded as the starting point for Medium-track training, and the Medium track best checkpoint was subsequently used as the starting point for Hard track training. The Hard curriculum run achieved 49 completed laps across 436 episodes (11.2% success), with a best reward of 651.0.

Greedy Evaluation

Each packaged best checkpoint was evaluated over 20 deterministic episodes (ε = 0) from the track's standard start position.

Track	Laps	Success	Collisions	Mean Reward	Mean Steps
Easy	20/20	100%	0	631.0	295
Medium	20/20	100%	0	576.0	241
Hard	20/20	100%	0	593.0	259

Training success includes exploratory random actions, while evaluation uses ε = 0. Medium and Hard use curriculum continuation from easier checkpoints; these results do not claim independent from-scratch convergence, arbitrary-start generalization, or unseen-track generalization.

User Interface and Model Reuse

The application has a Pygame-driven graphical user interface for interactive driving and reinforcement learning experiments. The menu consists of choices like Manual Drive, Train AI, Watch Trained AI, Evaluate, Settings, Reset, and Quit. The live interactive simulation shows the currently active episode, total reward, number of laps, speed of the vehicle, exploration rate (epsilon), and training losses during training and evaluation. Users have the option of changing the simulation speed of their models from 1x to 128x. The GUI helps users pause and resume the simulator, save the model immediately, switch on/off sensor rays view, and reset their trained model. Also, the difficulty of the track can be selected, and the vehicle color can be modified using red, blue, or green. Apart from that, the application saves the best and latest .pth checkpoints for replay or training resumption purposes. Each checkpoint has information about the learned policy, target network parameters, optimizer state, network architecture, training configuration, step counters, loss information, and necessary run metadata. Because of that, a previously trained model can be loaded for autonomous driving immediately without repeating the training process.

Testing and Reliability

The proposed system’s performance was assessed using a complete automated test suite built through pytest. The test cases focused on testing several primary components of the system, including track loading, validity of the track-mask, collision detection, vehicle dynamics (acceleration,

braking, steering, and friction), validity of the sensor ray-casting, and normalization of it. Besides, state and action interfaces were validated to ensure that the environment and the DQN agent cooperate correctly and find the correct state dimensionality as well as the four-action output space. Additionally, the automated test suite validated the reward computation process for various driving situations, completion of multiple laps, verification of the correct episode termination, dimensions of the DQN network and computations in the forward pass, validation of the replay buffer size and its operations, optimization, and loss computation procedures, and saving/loading checkpoints. Moreover, the end-to-end performance was tested to check if the model could make a successful lap without any accidents. The tests also covered the error handling aspects to ensure successful completion of the execution. All automated tests out of 20 executed showed good performance through multiple runs.

Discussion

The packaged checkpoints produce repeatable standard-start policies on Easy, Medium, and Hard from compact sensor inputs. Easy was trained directly via pure DQN with experience replay; Medium and Hard use curriculum continuation from easier checkpoints. Stability improved through several design choices: local progress tracking (preventing index-jump artifacts), ordered checkpoint gates (preventing reward oscillation), experience replay (breaking temporal correlation), a target network (stabilizing Bellman targets), gradient clipping (preventing divergence), and a deliberate separation between exploratory training ($\varepsilon > 0$) and greedy evaluation ($\varepsilon = 0$). Raw CSV and JSON outputs make the reported results inspectable rather than merely asserted.

Limitations

There are several drawbacks in the proposed system that need to be taken into account when evaluating the results of this system. First of all, random start generalization could not be verified as part of the packaged checkpoint evaluation since each track has the same starting pose. The vehicle dynamics are also highly simplified and do not consider such real world aspects as tire slip, suspension, and aerodynamics. Moreover, the state representation does not take into account any temporal memory systems such as LSTM networks and does not utilize any visual data provided by cameras or pixels. Results on Medium and Hard tracks are obtained by curriculum continuation from other tracks and not trained independently from scratch. Also, the control over the vehicle is constrained by the four-action discrete set that limits the precision of both acceleration and steering decisions.

Future Work

The future work may extend the approach suggested here in several ways for improved realism, control precision, and evaluation depth. The 2D environment can now be transported to a 3D simulation platform such as Unity, Unreal Engine, or CARLA with better vehicle dynamics like tire slip angles, suspension behavior, and weather dependent friction. The perception system can be moved from one dimensional ray cast sensors to raw camera images processed through Convolutional Neural Networks (CNN) for lane detection and obstacle detection functions. The discrete

four-action DQN may be substituted with more continuous actor-critic architectures such as Proximal Policy Optimization (PPO) or Soft Actor Critic (SAC) to achieve smoother control of the vehicle. The environment may also be enhanced with a multi agent traffic simulation with multiple dynamic cars. Future research may also provide an exhaustive comparison of from scratch training versus curriculum learning, application of various replay schemes like prioritized experience replay, and comparison of DQN with PPO and the like. Finally, a richer evaluation framework may be designed to randomize the initial positions, speeds, and headings of the cars, which may include model comparison dashboards and video recordings to ensure better evaluation of the learned driving policies.

Conclusion

This project connects the full reinforcement learning pipeline state, action, reward, replay, exploration, network update, checkpointing, and evaluation to a visible Pygame racing application. The included Easy, Medium, and Hard checkpoints and raw evaluation data provide reproducible evidence that a compact Double DQN with a 6 dimensional sensor state and 128 unit hidden layers can learn to complete each packaged track from its standard start pose on CPU, achieving a 100% success rate across all three difficulty levels with zero collisions in deterministic evaluation.

References

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, Feb. 2015.
[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
[3] PyTorch, “PyTorch documentation: Neural networks, optimization, and model serialization,” [Online]. Available: PyTorch Documentation
[4] Pygame, “Pygame documentation: Display, event, keyboard, surface, and drawing modules,” [Online]. Available: Pygame Documentation